ZADU.js - Measuring And Visualizing Distortions In Dimensionally Reduced Data On The Browser

A Thesis Presented
by

Jonathan Tarun Rajasekaran

to

The Department of Khoury College Of Computer Sciences

in partial fulfillment of the requirements
for the degree of

Master of Science

in

Computer Science

Northeastern University
Oakland, California

Apr 2026

NORTHEASTERN UNIVERSITY

Graduate School of Engineering

Thesis Title: ZADU.js - Measuring And Visualizing Distortions In Dimensionally Reduced Data On The Browser .

Author: Jonathan Tarun Rajasekaran.

Department: Khoury College Of Computer Sciences.

Approved for Thesis Requirements of the Master of Science Degree

Thesis Advisor: Dr. John A. Guerra-Gomez	Date
Thesis Reader: Dr. Yifan Hu	Date
Thesis Reader: Dr. Miguel Fuentes-Cabrera	Date
Department Chair:	Date

Director of the Graduate School:

Dean:	Date

Contents

List of Figures iv

List of Tables v

List of Acronyms vi

Acknowledgments viii

Abstract of the Thesis ix

1 Introduction 1
- 1.1 Problem Statement . . . 1
- 1.2 The Hyperparameter Problem . . . 2
- 1.3 The Case for Browser-Native Metrics . . . 3
- 1.4 The Inadequacy of Aggregate Statistics . . . 4
- 1.5 Contributions . . . 5
- 1.6 Thesis Outline . . . 5

2 Related Work 7
- 2.1 Quality Metrics for Dimensionality Reduction . . . 7
  - 2.1.1 Stress-Based Measures and Early Foundations . . . 7
  - 2.1.2 Trustworthiness, Continuity, and the Co-Ranking Matrix . . . 8
  - 2.1.3 Cluster-Level Metrics: Steadiness and Cohesiveness . . . 9
  - 2.1.4 The ZADU Library and Recent Surveys . . . 10
- 2.2 Visualization of DR Quality . . . 11
  - 2.2.1 CheckViz and Its Descendants . . . 11
  - 2.2.2 Other Visualization Approaches . . . 11
- 2.3 Hyperparameter Tuning and Interactive Exploration . . . 13
- 2.4 JavaScript Ecosystem for Dimensionality Reduction and In-Browser Machine Learning . . . 14
- 2.5 Positioning ZADU.js . . . 15

3 ZADU.js Library **17**
3.1 Trustworthiness: Detecting False Neighbors 17
3.2 Continuity: Detecting Missing Neighbors 18
3.3 Steadiness and Cohesiveness: Cluster-Level Metrics 19

4 MNIST Benchmark **21**
4.1 Benchmark Setup 21
4.2 Baseline Quality 22
4.3 Parameter Sensitivity Experiments 22
4.4 Per-Point Quality Map 24
4.5 Performance Benchmarks 25
4.6 Interactive Parameter Tuning Interface 26

5 Conclusion **32**

6 Future Work **34**

7 Appendix, Links, Code and Notebooks **36**

Bibliography **37**

List of Figures

4.1 UMAP baseline embedding on 200 MNIST samples with `n_neighbors` $= 15$ and `min_dist` $= 0.1$. 27
4.2 Color and size legend. 27
4.3 Metrics comparison table showing baseline and optimized scores with per-metric changes. 30
4.4 UMAP embedding recomputed with **min_dist** $= 0.08$ and **n_neighbors** $= 10$, colored using the viridis scale. 31

List of Tables

4.1 Baseline ZADU.js Quality Scores for MNIST-200 22
4.2 Directional Parameter Sweep Results on MNIST-200 24
4.3 Baseline vs. Best Configuration Found 25
4.4 Metric Computation Times on MNIST-200 ($n = 200$, $d = 784$) 25

7.1 ZADU.js Resources and Links 36

List of Acronyms

Co Cohesiveness. A cluster-level dimensionality reduction (DR) quality metric that measures whether groups of points that form coherent clusters in the original high-dimensional space remain spatially cohesive in the 2D embedding. Low Cohesiveness indicates that genuine clusters have been fragmented or dispersed.

C Continuity. A point-level DR quality metric that measures whether points that are close in the original high-dimensional space remain close in the 2D embedding. It captures the presence of missing neighbors, where genuine proximity relationships are not preserved.

DR Dimensionality Reduction. A class of techniques that transform high-dimensional data into a lower-dimensional representation, typically two or three dimensions, to enable visualization or simplify analysis while preserving important structural properties.

ES ECMAScript. The standardized specification that defines the core features of the JavaScript programming language. ES modules, introduced in ES2015, provide a native system for organizing and importing code across files.

HDBSCAN Hierarchical Density-Based Spatial Clustering of Applications with Noise. A density-based clustering algorithm that identifies clusters of varying density and labels outliers as noise. In ZADU.js, it is used to detect cluster structure for computing Steadiness and Cohesiveness.

PCA Principal Component Analysis. A linear dimensionality reduction technique that projects data onto orthogonal directions of maximum variance. PCA is often used as a preprocessing step or baseline method for comparison with nonlinear techniques.

S Steadiness. A cluster-level DR quality metric that measures whether clusters observed in the 2D embedding correspond to genuine cluster structure in the original high-dimensional data. Low Steadiness indicates the presence of phantom clusters.

SNN Shared Nearest Neighbor. A similarity or distance measure based on the number of shared nearest neighbors between two points. It is used in ZADU.js to assess cluster cohesion in high-dimensional space.

t-SNE t-Distributed Stochastic Neighbor Embedding. A nonlinear dimensionality reduction algorithm that emphasizes preservation of local neighborhood structure, often producing visually distinct clusters but with limited global structure fidelity.

T Trustworthiness. A point-level DR quality metric that measures whether points that appear close in the 2D embedding were genuinely close in the original high-dimensional space. It detects false neighbors introduced by the projection.

UI User Interface. The visual and interactive components of a software system through which users interact with the application, including controls, visualizations, and feedback elements.

UMAP Uniform Manifold Approximation and Projection. A nonlinear dimensionality reduction algorithm that constructs a graph representation of the data manifold and optimizes a low-dimensional embedding to preserve both local and some global structure.

VIS IEEE Visualization Conference. An annual international conference and the premier academic venue for research in data visualization, visual analytics, and related fields.

Acknowledgments

The completion of this thesis would not have been possible without the guidance, support, and encouragement of many individuals to whom I owe my deepest gratitude. First and foremost, I would like to express my sincere appreciation to my advisor, Dr. John Alexis Guerra-Gomez. His insight, patience, and thoughtful feedback shaped this work at every stage. I am especially grateful for his willingness to push me toward more rigorous thinking while giving me the freedom to explore my own ideas. His mentorship extended well beyond research — the lessons I take from our conversations will stay with me throughout my career. I would also like to thank the members of my thesis committee, Dr. Yifan Hu and Dr. Miguel Fuentes-Cabrera, for their time, careful review, and constructive criticism. Their questions challenged me to strengthen my arguments and refine my methodology in ways I could not have anticipated. My gratitude extends to the faculty and staff of Khoury College of Computer Sciences at Northeastern University, who created an environment in which curiosity and collaboration could thrive. I am also thankful to my fellow graduate students and labmates for the many conversations — academic and otherwise — that made this journey both intellectually richer and more enjoyable.

To my friends, thank you for the late-night encouragement, the well-timed distractions, and for reminding me that there is a world beyond the thesis. Finally, I want to thank my family. To my parents, for their unwavering belief in me and the sacrifices that made my education possible. To my sisters, thank you for your patience, your steady presence, and your faith in me on the days when my own was in short supply. This work is as much yours as it is mine.

Abstract of the Thesis

ZADU.js - Measuring And Visualizing Distortions In Dimensionally Reduced Data On The Browser

by

Jonathan Tarun Rajasekaran

Master of Science in Computer Science

Northeastern University, Apr 2026

Dr. John A. Guerra-Gomez, Adviser

We present ZADU.js, an open-source JavaScript library that computes per-point Trustworthiness (T) and Continuity (C) scores for dimensionality reduction (DR) embeddings entirely in the browser, without server infrastructure. DR projections such as UMAP introduce distortions, including false neighbors and missing neighbors, yet analysts rarely have tools that reveal where and how severely a given embedding misleads them. Existing DR quality frameworks depend on server-side Python environments, making them impractical for browser-based visual analytics systems. ZADU.js addresses this gap by providing per-point scores that can be mapped to visual channels such as color on a scatter plot, letting analysts localize distortion spatially rather than relying on a single aggregate statistic.

We benchmark the metrics on a 200-sample MNIST subset (784 dimensions to 2D), observing baseline scores of T = 0.93 and C = 0.95, and characterize the trade-offs that emerge across the parameter space. Metric computation completes in under 25ms for point-level metrics and under 250ms for cluster-level metrics on this dataset, establishing the feasibility of browser-native, interactive quality assessment.

Chapter 1

Introduction

1.1 Problem Statement

Dimensionality reduction has become a standard preprocessing step in data analysis and visualization. Algorithms such as UMAP [23], t-SNE, and PCA are routinely applied to datasets spanning hundreds or thousands of dimensions in order to produce two-dimensional scatter plots that analysts can visually inspect for clusters, trends, and outliers. These projections are ubiquitous: they appear in genomics, natural language processing, image analysis, and information visualization, where researchers use them to form hypotheses about structure in complex data.

The intuitive appeal of these projections carries a significant and underappreciated risk. Every projection necessarily discards information - compressing, for example, 784 dimensions of handwritten digit pixel data into just 2 coordinates. The resulting layout implies geometric relationships that may or may not reflect the actual structure of the original data. A compact cluster in the scatter plot suggests that those points are genuinely similar. Two well-separated groups suggest that they are genuinely distinct. Nearby points are assumed to be genuine neighbors. All of these inferences can be wrong.

Consider an analyst who examines a scatter plot of gene expression data and identifies two apparent cell populations, drawing biological conclusions from their visual separation. If those populations were in fact closely intermingled in the original 100-dimensional expression space, the conclusion is an artifact of the projection - a phantom cluster created by

the mathematics of dimensionality reduction - rather than a biological finding. The analyst had no way to know this from the scatter plot alone.

The critical question is not whether distortion is present - it always is - but which relationships are distorted and where. Distortion that is uniformly distributed and small may be acceptable. Distortion that is concentrated in a specific region of the scatter plot is dangerous, because that region will be misinterpreted by any analyst who trusts the visual layout.

Despite this risk, the tools available to practitioners for assessing embedding quality are limited by two practical barriers. First, existing quality assessment libraries, such as the Python ZADU library [11] and CheckViz [19], operate server-side and require Python environments, making them inaccessible within browser-based visualization workflows. Second, even when quality metrics are computed, they are typically reported as a single global score - a Kruskal stress value or a global Trustworthiness score - that cannot reveal where the distortion is located.

ZADU.js addresses this problem by making the effects of each parameter choice measurable. Quality scores computed after a UMAP run reveal how the projection has changed along multiple axes of structural faithfulness, letting the analyst compare configurations on data-grounded criteria rather than visual judgment alone.

1.2 The Hyperparameter Problem

UMAP exposes hyperparameters that significantly affect embedding quality, and selecting them is a persistent practical challenge for both researchers and practitioners. The two most influential are **n_neighbors** and **min_dist**.

The **n_neighbors** parameter controls the size of the local neighborhood used to construct the manifold approximation graph. Small values (5–10) make UMAP focus on very local structure, producing tight, well-separated clusters that capture fine-grained proximity but may sacrifice global layout, specifically the relative positioning of clusters from one another. Large values (30–50) incorporate more global context, improving the inter-cluster relationships visible in the scatter plot at the expense of fine local detail and within-cluster

cohesion.

The **min_dist** parameter controls the minimum distance between points in the 2D embedding. Small values produce compact, tightly packed clusters, while large values allow more diffuse, continuous layouts where points within a cluster are spread out. This parameter interacts strongly with **n_neighbors**. The two parameters jointly determine both what constitutes a neighborhood during manifold learning and how compactly that neighborhood is rendered in the embedding.

No universally optimal values exist for these parameters. Their effect is dataset-dependent, varying with sample size, intrinsic dimensionality, the degree of class overlap in the original space, and the analyst's priority, whether local or global structure. In practice, users iterate by visual inspection alone, repeatedly running UMAP with different parameters and judging the resulting scatter plots subjectively. This is enhanced by the interactive hyperparameter tuning application that uses the metrics from the ZADU.js library to inform their decisions.

1.3 The Case for Browser-Native Metrics

Browser-based visual analytics tools (interactive dashboards, exploratory notebooks, live demonstrations) are increasingly the primary interface between DR output and the analyst. Observable notebooks, Vega-Lite dashboards, and custom D3.js visualizations bring data exploration directly into the browser, eliminating the installation overhead of desktop applications. Users can share a URL and immediately explore a live, interactive visualization without configuring an environment.

Existing quality assessment tools break this model. The Python ZADU library requires a Python environment with NumPy, SciPy, and scikit-learn. CheckViz requires matplotlib and server-side computation. Integrating either into a browser-based visualization requires a backend server, an authentication boundary, a network round-trip per computation, and ongoing infrastructure maintenance. For exploratory analysis or rapid prototyping, this overhead makes quality assessment practically inaccessible.

Running DR and quality metrics entirely in the browser eliminates this friction. The

JavaScript ecosystem for browser-native computation has matured sufficiently to support this: druid.js [7] provides a browser-native UMAP implementation, and Transformers.js [20] enables running pre-trained transformer models in-browser via WebAssembly, making it possible to compute sentence embeddings on the client side. Quality assessment was the missing piece. ZADU.js completes the pipeline: data enters the browser, DR runs in the browser, quality is measured in the browser, and the annotated scatter plot is rendered in the browser - with no server involved at any step.

This architecture has practical implications beyond convenience. Browser-native quality assessment can be made interactive and real-time: as the analyst adjusts UMAP parameters and re-computes the embedding, quality scores can be recomputed and overlaid without any network latency. The entire feedback loop (embed, assess, adjust, re-embed) fits within a single browser session.

1.4 The Inadequacy of Aggregate Statistics

Current practice often reports a single global quality number, such as Kruskal stress or a global Trustworthiness score. These aggregates are necessary but insufficient. A global T of 0.93 says that approximately 7% of apparent proximity relationships in the scatter plot are artifacts, meaning points that appear close but were not genuinely close in the original space. However, it does not indicate where those artifacts are, or whether they are concentrated in a single misleading region or uniformly distributed across the plot.

An analyst who encounters a suspicious cluster in a scatter plot with $T = 0.93$ cannot determine from that number alone whether the cluster reflects real structure or is a phantom artifact of the projection. To make that determination, the analyst needs to know the per-point T scores for the points in that cluster. If those points all have low individual T scores, the cluster is likely artifactual. If their T scores are high, the cluster is likely genuine.

Per-point scores provide this localization. By mapping individual T and C values to a color scale, for example green for high quality and red for low quality, ZADU.js transforms a single scalar into a spatial distortion map layered onto the scatter plot. Regions of high distortion become visually salient. The analyst can immediately identify which

parts of the projection to trust and which to question, without any additional computation or exploration.

This approach is consistent with the broader principle in information visualization that uncertainty and reliability information should be displayed in context, at the point where it is relevant to interpretation, rather than reported separately as an aggregate [3].

1.5 Contributions

This work makes the following contributions:

1. ZADU.js: An open-source JavaScript implementation of four complementary DR quality metrics, Trustworthiness (T), Continuity (C), Steadiness (S), and Cohesiveness (Co), with per-point local scores, compatible with UMAP and any other DR algorithm that produces a 2D embedding. The library is available on npm and as a CDN import.

2. Interactive Hyperparameter Tuning Application: A self-contained browser application that lets analysts adjust UMAP parameters with sliders, re-run the projection in the browser, and immediately compare the resulting embedding against a baseline using side-by-side scatter plots colored by per-point quality score.

3. Browser-native deployment: The entire pipeline, including metric computation, visualization, and optimization, runs as ES modules with no server, no installation, and no Python dependency, making it directly usable in any JavaScript visualization pipeline.

1.6 Thesis Outline

Chapter 2, Related Work: Surveys existing approaches to DR quality assessment, quality visualization, and the JavaScript ecosystem for dimensionality reduction, situating ZADU.js within the broader research landscape.

Chapter 3, ZADU.js Library: Describes the four quality metrics in detail, including their mathematical formulations, interpretation, and implementation as ES modules, and presents the per-point visualization module and library architecture.

Chapter 4, MNIST Benchmark: Reports quality scores, parameter-sensitivity results, and computation time measurements on a 200-sample MNIST subset, providing a controlled characterization of metric behavior on a well-understood dataset.

Chapter 5, Conclusion: Summarizes the contributions and findings of the thesis.

Chapter 6, Future Work: Outlines directions for extending ZADU.js, including scalability improvements, support for additional DR algorithms, and a formal usability study.

Chapter 2

Related Work

The reliability of visual analytics grounded in dimensionality reduction rests on three interlocking bodies of work: the development of quality metrics that quantify how faithfully a projection preserves structure, the visualization of those metrics so that distortion becomes visible rather than merely measurable, and the engineering of libraries that make both computation and visualization accessible to practitioners. ZADU.js sits at the intersection of these threads, porting a mature metric framework into the browser ecosystem and integrating it with an iterative parameter optimization loop. The sections that follow review the most relevant prior work across these areas. We also survey recent developments in hyperparameter tuning for projection methods, and we characterize the JavaScript machine-learning ecosystem that makes a browser-native quality library feasible today.

2.1 Quality Metrics for Dimensionality Reduction

2.1.1 Stress-Based Measures and Early Foundations

Quantifying how faithfully a low-dimensional embedding preserves the structure of high-dimensional data is a problem with roots in multidimensional scaling (MDS). Torgerson [29] introduced classical metric MDS as a method for recovering configurations of points from matrices of pairwise proximities, and Kruskal [16] generalized this to non-metric MDS by introducing the stress function, a normalized sum of squared discrepancies

between original and projected pairwise distances. Kruskal stress has remained the canonical global distortion measure for more than six decades because it is simple, intuitive, and easy to compute. Recent work by Smelser et al. and Kobourov's group, however, has shown that normalized stress is sensitive to uniform scaling of the projection, even though such scaling does not meaningfully change the projection's information content, motivating scale-invariant variants of the measure.

2.1.2 Trustworthiness, Continuity, and the Co-Ranking Matrix

Venna and Kaski [31, 32] introduced Trustworthiness and Continuity as complementary measures of local neighborhood preservation, and Kaski et al. [15] refined and popularized them. Trustworthiness penalizes points that appear close in the embedding but were not genuinely close in the original space (false neighbors, an analog of precision in information retrieval). Continuity penalizes the symmetric error: points that were close in the original space but have been pushed apart in the embedding (missing neighbors, an analog of recall). Kaski et al. argued that trustworthiness is often the more important of the two for visualization because visualized proximities are especially salient to the viewer, who is prone to read spatial closeness as genuine similarity. Together, T and C characterize two distinct ways the neighborhood structure can be corrupted, and both must be high for the local reading of a scatter plot to be reliable.

Lee and Verleysen [17, 18] unified Trustworthiness, Continuity, and related rank-based criteria within the co-ranking matrix framework. The co-ranking matrix Q has entries q_{kl} counting the number of point pairs whose ranks change from k in the original space to l in the embedding. Different quality criteria can be expressed as weighted sums over characteristic blocks of Q. The upper-left $K \times K$ block measures preserved neighborhoods, known as the Local Continuity Meta-Criterion of Chen and Buja [5]. The off-diagonal blocks measure intrusions and extrusions, and T and C correspond to specific weighted block sums. The framework allows principled comparison of rank-based measures across scales of K, and it supplies a theoretical basis for simple aggregated criteria such as the $Q_{NX}(K)$ score, whose area under the curve under a logarithmic K scale became a widely adopted single-number summary [17]. Lueks et al. [21] subsequently proposed refinements

that weight the magnitude of rank errors in addition to their frequency, addressing a key weakness of the original formulation.

Venna et al. [33] reframed quality assessment explicitly as an information retrieval problem, formalizing the precision-recall analogy and proposing NeRV (Neighbor Retrieval Visualizer), a DR method that directly optimizes a weighted combination of T and C. This information-theoretic perspective has since shaped many subsequent contributions. Cat-SNE and ClassNeRV are class-aware extensions that optimize neighborhood preservation while taking label information into account, improving classification performance in the embedding without sacrificing structural fidelity.

2.1.3 Cluster-Level Metrics: Steadiness and Cohesiveness

Trustworthiness and Continuity operate at the point level and are fundamentally blind to higher-order cluster structure. A projection can score well on T and C while still creating phantom clusters or shattering real ones, because neither measure examines group-level arrangement. Jeon et al. [12] introduced Steadiness and Cohesiveness as cluster-level analogs, addressing precisely this gap. Steadiness asks whether visual clusters in the embedding correspond to genuine cluster structure in the original space, penalizing false groups created by the projection. Cohesiveness asks the symmetric question, penalizing missing groups where genuine clusters have been dispersed across the scatter plot. The computation uses HDBSCAN density-based clustering and random walk sampling, with pairwise distances among members computed using a Shared Nearest Neighbor metric to handle the curse of dimensionality in the high-dimensional space. Jeon et al. also introduced a complementary visual artifact, the reliability map, which encodes pointwise Steadiness and Cohesiveness scores onto the scatter plot using a bivariate color scheme inspired by CheckViz. The publication of Steadiness and Cohesiveness filled an important gap in the metric landscape because cluster analysis is one of the most common tasks analysts perform on projected data. The authors empirically demonstrated that T and C are insensitive to several classes of inter-cluster distortions that Steadiness and Cohesiveness do detect, confirming that point-level and cluster-level metrics are complementary rather than redundant. ZADU.js incorporates both metric families in order to provide this com-

plementary coverage.

2.1.4 The ZADU Library and Recent Surveys

Jeon et al. [11] packaged seventeen distortion measures into the ZADU Python library, and the maintained version of the library now provides twenty measures. ZADU reuses existing open-source implementations where available and reimplements the remainder, applying a unified optimization pipeline so that preprocessing steps such as pairwise distance computation and kNN identification are shared across metrics. A companion library, ZADUVis, renders the bivariate CheckViz-style map and the reliability map from pointwise distortion scores. ZADU.js is a direct JavaScript port of the metric subset most relevant to scatter-plot analysis (Trustworthiness, Continuity, Steadiness, and Cohesiveness) and a browser-native rendering pipeline. Jeon et al. [14] subsequently published a comprehensive survey of 133 papers on the reliability of visual analytics with DR, building a taxonomy of reliability problems across the visual analytics workflow (preprocessing, dimensionality reduction, quantitative evaluation, visualization, sensemaking) and a corresponding taxonomy of solutions. Six functional clusters of systems emerge from their meta-analysis (Pioneer, Judge, Instructor, Explorer, Explainer, Architect), and the survey identifies metric approachability, particularly in non-Python environments, as an ongoing challenge. This is precisely the gap that ZADU.js is designed to close. Earlier surveys by Nonato and Aupetit [24], Espadoto et al. [8], and Van der Maaten et al. [30] provide complementary taxonomies of DR techniques themselves, their distortions, and their empirical performance across datasets. Espadoto et al. in particular present a large-scale empirical comparison of 44 projection techniques across 18 datasets using multiple quality metrics, serving as a reference benchmark for the field. Recent work also scrutinizes widely used metrics. Saturn, proposed by Chicco et al., claims to provide a simpler alternative to T and C for UMAP evaluation. Kobourov and collaborators have highlighted that normalized stress varies with projection scaling, and have proposed scale-invariant corrections. Graving and Couzin's VAE-SNE benchmarking, the GhostUMAP work by Jeon et al. on pointwise instability, and Atzberger et al.'s large-scale sensitivity analysis on text embeddings all emphasize that metric choice, hyperparameter choice, and stochastic initialization

interact in ways that aggregate numbers can easily obscure.

2.2 Visualization of DR Quality

2.2.1 CheckViz and Its Descendants

CheckViz [19], introduced by Lespinats and Aupetit , is the most directly related prior work to ZADU.js in quality visualization. It provides a visual interface for inspecting pointwise distortion on a scatter plot using a two-dimensional perceptually uniform color coding that simultaneously represents false neighborhoods and tears (missing neighborhoods), the two elementary types of geometric mapping distortion. CheckViz belongs to the Verity Visualization framework, which emphasizes that any inference from a projection back to the multidimensional space requires first quantifying the mapping quality and exposing it to the viewer. CheckViz is implemented in Python using matplotlib and scipy, however, and requires server-side computation. It is also not actively maintained, and no JavaScript port has been released. ZADU.js replicates the core CheckViz concept, color-encoding pointwise quality onto the scatter plot, while adding four complementary improvements. First, it runs natively in the browser without a server. Second, it covers four metrics rather than two, extending the coverage from point-level neighborhood preservation to cluster-level structure preservation. Third, it is distributed as an npm package with a simple array-based API that integrates directly with existing JavaScript visualization pipelines.

2.2.2 Other Visualization Approaches

Several other visualization techniques for DR quality have been proposed, each emphasizing a different aspect of distortion. Stress Maps [28] compute pointwise stress values and render them as a map-like overlay, highlighting regions where pairwise distance preservation is poor. Aupetit's probing method [3] detects local distortions by analyzing how small perturbations in the high-dimensional space manifest in the projection, using Voronoi cells over the embedding to assign each pixel a local distortion estimate. Martins et al. [22] proposed visual approaches that use Voronoi diagrams and color mappings

to communicate neighborhood preservation spatially, as well as techniques for explaining which dimensions or attributes contribute most to observed distortions. Colange et al. [6] introduced MING (Map Interpretation using Neighbourhood Graphs), which superimposes nearest-neighbor graphs onto the scatter plot to expose the discrepancy between visual and actual proximity. MING serves as a continuous reminder to the viewer that the map is not the data, directly addressing the interpretation uncertainty that Aupetit formalized in his often-cited observation that a projection is neither true nor false but may be faithful or not to some geometric or topological criterion. Jeon et al.'s reliability map [12] is specifically tailored to Steadiness and Cohesiveness, using a bivariate colormap to encode pointwise false-group and missing-group distortion, and providing a lasso interaction that highlights points belonging to the same original-space cluster. The reliability map is a React component that renders client-side but still requires server-side Python computation of the metrics, so it does not qualify as an end-to-end browser-native solution. ZADU.js's per-point visualization borrows the spirit of this approach while computing all scores on the client. The Warping Index [26] quantifies global projection distortion by measuring how uniformly the high-dimensional space maps onto the 2D layout, penalizing regions where the projection compresses or expands distances unevenly. While it provides a useful aggregate signal of global structure preservation, it does not decompose scores to the point level, making it impossible to spatially localize which regions of the scatter plot are most distorted. Hypertrix [25] takes a fundamentally different approach: rather than evaluating the embedding, it analyzes the geometric structure of the high-dimensional data itself, identifying regions where the data's intrinsic dimensionality makes faithful 2D projection inherently impossible. This framing is complementary to T and C because it characterizes properties of the input data and the projection transformation rather than the embedding output. Neither the Warping Index nor Hypertrix produces per-point scores that can be mapped to visual channels on the scatter plot, nor does either tool provide a browser-native implementation. Several systems go beyond static per-point maps and combine metric computation with rich interactive analysis. DimReader [9] recovers readable axes from non-linear projections by analyzing infinitesimal perturbations of the dataset, providing interpretive guidance that is otherwise lost when the projection is non-parametric. Distortion-aware Brushing [10] allows the analyst to select clusters in the projected space while taking into account the dis-

tortions that the metrics quantify, so that the brushed set corresponds to a coherent region of the original high-dimensional space rather than a coincidental cluster in 2D. Explainers such as LayerFlow extend these ideas into the space of LLM embeddings and large models. These systems are all richer than what ZADU.js offers in isolation, but they are also heavier, domain-specific, and typically Python-bound. The design goal of ZADU.js is instead a lightweight, composable library that any JavaScript visualization can adopt with minimal friction.

2.3 Hyperparameter Tuning and Interactive Exploration

UMAP's behavior is governed principally by two hyperparameters, **n_neighbors** and **min_dist**, whose values interact nonlinearly with dataset size, intrinsic dimensionality, and the analyst's priorities [23]. Hyperparameter tuning for projection methods has therefore become a sub-field of its own.

HyperNP [2] addresses the hyperparameter problem directly by training a neural network approximator that maps (data point, hyperparameter value) pairs to 2D coordinates, enabling real-time interactive exploration of the hyperparameter space without recomputing the full projection at each setting. HyperNP is fast enough to embed in a browser-based visualization system, making it directly relevant to the browser-native setting that ZADU.js targets. The two approaches are complementary: HyperNP accelerates the exploration of hyperparameter space by approximating the projection, while ZADU.js quantifies the quality of any given projection, letting analysts make informed parameter choices on measurable grounds rather than visual inspection alone.

Bayesian optimization frameworks have also been applied to DR hyperparameter selection, using surrogate models to efficiently search multi-objective trade-off surfaces. These methods typically run offline and target a single optimum. ZADU.js is complementary to these approaches: rather than automating hyperparameter search, it provides the quality measurements that any search strategy — automated or human-driven — needs to make informed choices.

At the level of DR algorithm design rather than tuning, several methods have been pro-

posed specifically to address the local-global trade-off that vanilla UMAP and t-SNE exhibit. TriMap [1] uses triplet constraints to preserve global relationships, and PaCMAP [34] introduces mid-near pairs to capture global structure before refining local detail, demonstrating in large-scale benchmarks that both local and global preservation can be achieved simultaneously. UMATO [13] modifies UMAP's optimization to bridge local and global structure preservation more reliably. These algorithms change the object being projected rather than the projection's parameters, and quality metrics like those in ZADU.js are necessary to evaluate them on common ground. ZADU.js is algorithm-agnostic and can assess any projection technique that produces a 2D embedding, so it integrates naturally with the broader ecosystem of DR methods.

2.4 JavaScript Ecosystem for Dimensionality Reduction and In-Browser Machine Learning

The JavaScript ecosystem for dimensionality reduction has matured considerably in recent years, creating the conditions under which a browser-native quality assessment library becomes both feasible and valuable. Several projection implementations have appeared in the JavaScript ecosystem over the past decade. An early t-SNE port (tSNEjs) exposed a functional but low-level API that required the developer to manage iteration manually. MachineLearn.js provided PCA among a broader set of ML primitives. The umap-js package provides a standalone UMAP implementation widely used in observable notebooks and web applications. The druid.js library [7], distributed on npm as @saehrimnir/druidjs, consolidates ten DR algorithms including PCA, MDS, t-SNE, UMAP, Isomap, LLE, LTSA, FastMap, and others, under a clean ES module API that mirrors Scikit-learn's fit/transform convention. Druid.js is dependency-free, trivially embeddable in any JavaScript project, and provides a consistent programming interface across methods. Its UMAP implementation follows a slightly newer formulation than the original 2018 McInnes et al. paper. ZADU.js is designed to interoperate with druid.js: it accepts embeddings produced by druid.js, or by any other source, as plain JavaScript arrays, requiring no coupling to a specific DR implementation. The combination of druid.js for the projection and ZADU.js for

quality assessment provides a complete browser-native pipeline that parallels the Python scikit-learn + ZADU stack. Beyond DR itself, the browser ecosystem for machine learning has expanded dramatically with WebAssembly-based inference engines. TensorFlow.js and ONNX.js brought server-side models to the client, and Transformers.js [20] built on ONNX Runtime to run pretrained transformer models, including BERT, DistilBERT, T5, CLIP, and sentence transformers, entirely in the browser. Recent versions support WebGPU acceleration, enabling state-of-the-art models to run at practical speeds on consumer hardware. Visualization libraries such as D3.js [4] and Vega-Lite [27] provide the rendering infrastructure for interactive scatter plots. D3's data-join model and Vega-Lite's declarative grammar both integrate naturally with the array-based outputs that ZADU.js produces. Quality maps produce plain arrays of pointwise color and score data aligned with the input, so developers can layer quality information onto existing scatter plots with minimal additional code. The library's output format, parallel arrays of scores and hex color strings, was designed specifically with D3 and Vega-Lite in mind. Observable notebooks have emerged as a popular medium for sharing interactive DR demonstrations, and ZADU.js ships with an Observable notebook that imports the library from npm, runs a UMAP projection on a 200-sample MNIST subset using druid.js, and evaluates it with Trustworthiness and Continuity. The notebook requires no installation and demonstrates the friction-free deployment story that is central to the library's value proposition.

2.5 Positioning ZADU.js

ZADU.js differs from all of the approaches reviewed above along four dimensions. First, it runs natively in the browser without a server, without a Python environment, and without any installation beyond a single npm import. Second, it computes per-point scores that can be interactively queried and color-encoded in real time, exposing the spatial distribution of distortion that aggregate statistics hide. Third, it covers four metrics, Trustworthiness, Continuity, Steadiness, and Cohesiveness, providing both point-level and cluster-level quality assessment in a single library. Fourth, it integrates directly with JavaScript DR implementations and rendering libraries through a simple array-based API, enabling devel-

opers to add quality assessment to an existing scatter plot in a few lines of code. Taken together, these contributions fill the browser-native gap in the DR quality assessment literature identified by Jeon et al.'s 2025 survey.

ZADU.js is an open-source JavaScript library for measuring the quality of dimensionality reduction embeddings directly in the browser. It takes two inputs (a high-dimensional dataset and a 2D embedding produced by any DR algorithm) and returns a set of quality scores at both the global and per-point level. No server, no Python environment, and no preprocessing pipeline are required.

The library's design philosophy prioritizes integration over isolation. Quality scores are returned as plain arrays aligned with the input data, so they can be mapped directly to visual channels (color, opacity, radius) in any scatter plot implementation that accepts per-point styling. A developer who already has a UMAP scatter plot rendered with D3.js can add per-point quality coloring in fewer than ten lines of additional code.

Chapter 3

ZADU.js Library

3.1 Trustworthiness: Detecting False Neighbors

Trustworthiness measures the degree to which apparent proximity relationships in the embedding reflect genuine proximity in the original space. Intuitively, it asks: for each point i, are the points that appear close to it in the scatter plot genuinely close to it in the original high-dimensional space?

For each point i, the computation identifies the set $\tilde{U}_k(i)$, the set of points that are among i's k nearest neighbors in the embedding but whose rank in the original high-dimensional space exceeds k. These are the false neighbors: points that appear close in the visualization but were not genuinely close in the data.

Formal definition:

$$T(k) = 1 - \frac{2}{n \cdot k \cdot (2n - 3k - 1)} \sum_{i=1}^{n} \sum_{j \in \tilde{U}_k(i)} (r(i,j) - k) \tag{3.1}$$

where $r(i,j)$ is the rank of point j among i's neighbors in the original space, and n is the total number of points. The penalty for each false neighbor is weighted by how far its original-space rank exceeds k. A point that appears as a second-nearest neighbor in the embedding but was the 500th-nearest neighbor in the original space incurs a much larger penalty than one that was the 15th-nearest neighbor. The normalization factor ensures the score lies in $[0, 1]$, with 1 indicating that no false neighbors are introduced.

Per-point scores: The global Trustworthiness score aggregates penalties across all n points. ZADU.js also returns the per-point penalty sum for each point i before normalization. These per-point scores form the basis of the quality map described in Section 4.4. Points with high per-point penalties, corresponding to low T scores, are those whose apparent position in the embedding is most misleading.

3.2 Continuity: Detecting Missing Neighbors

Continuity is the symmetric complement of Trustworthiness. While T asks whether points close in the embedding were close in the original space, C asks whether points close in the original space remain close in the embedding. It addresses the question: are any genuine proximity relationships being hidden by the projection?

For each point i, the computation identifies the set $V_k(i)$, the set of points that were among i's k nearest neighbors in the original high-dimensional space but whose rank in the embedding $\hat{r}(i,j)$ exceeds k. These are the missing neighbors: points that were genuinely nearby but have been pulled apart in the visualization.

Formal definition:

$$C(k) = 1 - \frac{2}{n \cdot k \cdot (2n - 3k - 1)} \sum_{i=1}^{n} \sum_{j \in V_k(i)} (\hat{r}(i,j) - k) \tag{3.2}$$

where $\hat{r}(i,j)$ is the rank of point j among i's neighbors in the embedding. The formula is structurally identical to Trustworthiness, with the roles of original and embedding space swapped.

Interpretation: Together, T and C characterize two distinct failure modes of a DR projection. A low T score indicates false closeness, where the scatter plot shows a proximity relationship that does not exist in the data. A low C score indicates false separation, where the scatter plot hides a proximity relationship that does exist in the data. Both must be high for the local neighborhood structure of the visualization to be reliable. An analyst should treat regions of low T as potentially showing phantom proximity, and regions of low C as potentially obscuring genuine proximity.

3.3 Steadiness and Cohesiveness: Cluster-Level Metrics

Trustworthiness and Continuity operate at the point level and evaluate local neighborhood relationships. They are, by design, blind to higher-order cluster structure. A projection can have high T and C while still creating phantom clusters, which are visual groupings that do not correspond to genuine clusters in the original space, or dispersing genuine clusters across the scatter plot. Steadiness and Cohesiveness address these cluster-level failure modes.

Steadiness: Steadiness measures whether the visual clusters apparent in the embedding correspond to genuine clusters in the original space. The computation proceeds in three steps:

1. HDBSCAN density-based clustering is applied to the 2D embedding to identify visually apparent groups.

2. For each identified cluster, a random walk process extracts a representative set of member points, sampling the cluster's internal structure.

3. Pairwise distances among the sampled members are computed in the original high-dimensional space using a Shared Nearest Neighbor (SNN) distance metric. If members that appear clustered in the embedding are in fact scattered in the original space, a penalty is assessed.

A low Steadiness score indicates that one or more of the embedding's visual clusters is a phantom. The cluster structure exists in the scatter plot but not in the data. This is the cluster-level analog of a false neighbor.

Cohesiveness: Cohesiveness is the cluster-level symmetric complement of Steadiness. Rather than identifying clusters in the embedding, HDBSCAN is applied to the original high-dimensional data to identify genuine cluster structure. The computation then measures whether each identified cluster's members remain spatially coherent in the 2D embedding. A low Cohesiveness score means that genuine clusters in the data have been dispersed across the scatter plot, split into multiple visual groups or merged with unrelated points, making them difficult to identify visually.

The S–Co trade-off. Like T and C, Steadiness and Cohesiveness exhibit a fundamental trade-off, but one with a different character. A large S–Co gap, or both metrics simultaneously below an acceptable threshold, is a diagnostic signal that the dataset's cluster structure partially overlaps in the original high-dimensional space. When digit classes 4 and 9 are geometrically entangled in 784-dimensional pixel space, no 2D projection can simultaneously keep them in separate, coherent clusters, corresponding to high Co, without also risking the creation of phantom cluster boundaries, corresponding to low S. ZADU.js detects this condition and reports it explicitly, rather than continuing to optimize indefinitely, correctly informing the analyst that the problem is a property of the data rather than a correctable parameter choice.

Chapter 4

MNIST Benchmark

4.1 Benchmark Setup

We evaluate ZADU.js on a 200-sample subset of the MNIST handwritten digits dataset, projecting from 784 dimensions (28×28 pixel intensity arrays, flattened) to 2D using UMAP via the `druid.js` library. The sample consists of 20 images per digit class (0–9), selected uniformly at random. Default UMAP parameters are used for the baseline: **n_neighbors** $= 15$, **min_dist** $= 0.10$, $n_epochs = 350$, and $seed = 42$.

This benchmark serves two purposes. First, it provides a controlled characterization of metric behavior on a dataset with well-understood ground-truth structure. MNIST digit classes are known to be partially separable in pixel space, with certain digit pairs, such as 4 and 9 or 3 and 8, being geometrically similar. This prior knowledge allows us to interpret metric scores in terms of known data properties rather than inferring properties from the scores alone. Second, the small sample size ($n = 200$) allows all computations, including the more expensive Steadiness and Cohesiveness metrics, to run in the browser within seconds, making the benchmark interactive.

The neighborhood parameter k is set to 7 for Trustworthiness and Continuity, and to the default of $\sqrt{n} \approx 14$ for Steadiness and Cohesiveness. All experiments were run in a Node.js environment for reproducibility. The same code runs in any modern browser.

4.2 Baseline Quality

At default UMAP parameters, ZADU.js reports the scores shown in Table 4.1.

Table 4.1: Baseline ZADU.js Quality Scores for MNIST-200

Metric	**Score**
Trustworthiness (T)	0.9281
Continuity (C)	0.9494
Steadiness (S)	0.7913
Cohesiveness (Co)	0.6432
Composite (average)	0.828

The high T and C scores indicate that local neighborhood structure is well preserved at the default parameters. Few false neighbors are introduced ($T = 0.9281$) and few genuine neighbors are separated ($C = 0.9494$). The notably lower S and Co scores reflect the inherent challenge of projecting 784-dimensional digit data to 2D, and are consistent with what is known about MNIST's geometric properties.

The Cohesiveness score of 0.6432 indicates that several genuine digit classes are geometrically entangled in the 2D projection. Specifically, the visually similar pairs 4 and 9, and 3 and 8, are known to occupy overlapping regions of pixel space. The curves and strokes that characterize these digit classes are geometrically similar enough that UMAP cannot cleanly separate them in 2D without distorting other relationships. The Steadiness score of 0.7913 reflects a related phenomenon. Some visual clusters in the embedding, where the algorithm creates apparent groupings, do not correspond cleanly to individual digit classes in the original pixel space.

4.3 Parameter Sensitivity Experiments

The parameter sensitivity experiments were run directly via Node.js for reproducibility. The UMAP implementation used is umap-js rather than druid.js, and no fixed random seed was applied, so the absolute metric values differ slightly from Table 4.1. The qualitative findings are consistent across both implementations.

A structured directional sweep was conducted from the baseline (**n_neighbors** $= 15$, **min_dist** $= 0.10$) across four search directions, five steps each, for 20 total configurations. The four directions are:

- **Direction A:** both parameters decrease ($n_neighbors$ down 5, min_dist down 0.02 per step)
- **Direction B:** both parameters increase ($n_neighbors$ up 5, min_dist up 0.02 per step)
- **Direction C:** $n_neighbors$ increases, min_dist decreases
- **Direction D:** $n_neighbors$ decreases, min_dist increases

Table 4.2 shows the full sweep results. The baseline composite score was 0.846.

Direction A is uniformly the worst. Reducing **n_neighbors** below 15 degrades all metrics, with Cohesiveness dropping as low as 0.548. Direction D collapses after the first step because **n_neighbors** reaches its lower bound. Direction C produces modest gains but fails to substantially improve Cohesiveness.

Direction B is the clear winner. Configurations B1 and B2 achieve the highest composite score of 0.853, with Cohesiveness improving to 0.670 and 0.681 respectively. Increasing **min_dist** allows points to spread out in the embedding, reducing compression and overlap between clusters. Combined with a larger neighborhood, this improves global structure representation at the cost of slightly reduced cluster sharpness.

The best configuration improves the composite score by 0.7 percentage points and raises Cohesiveness by 6.1 percentage points, at a small cost to T, C, and S. This reflects the inherent S–Co trade-off. Neither configuration fully resolves the Cohesiveness deficit, which is caused by geometric overlap of digit classes in the original high-dimensional space.

The practical takeaway is that increasing both **n_neighbors** and **min_dist** together is an effective strategy when Cohesiveness is the limiting metric.

Table 4.2: Directional Parameter Sweep Results on MNIST-200

Run	**n_neighbors**	**min_dist**	T	C	S	Co	**Composite**
Baseline	15	0.10	0.941	0.955	0.868	0.620	0.846
A1	10	0.08	0.939	0.953	0.850	0.593	0.834
A2	5	0.06	0.921	0.927	0.809	0.548	0.801
A3	5	0.04	0.930	0.931	0.841	0.554	0.814
A4	5	0.02	0.927	0.929	0.847	0.558	0.815
A5	5	0.01	0.911	0.925	0.805	0.580	0.805
B1	20	0.12	0.932	0.955	0.854	0.670	0.853
B2	25	0.14	0.938	0.950	0.844	0.681	0.853
B3	30	0.16	0.915	0.950	0.832	0.668	0.841
B4	35	0.18	0.914	0.951	0.811	0.654	0.833
B5	40	0.20	0.915	0.948	0.809	0.688	0.840
C1	20	0.08	0.929	0.953	0.846	0.635	0.841
C2	25	0.06	0.938	0.954	0.842	0.670	0.851
C3	30	0.04	0.939	0.951	0.841	0.662	0.848
C4	35	0.02	0.935	0.953	0.834	0.650	0.843
C5	40	0.01	0.933	0.951	0.830	0.662	0.844
D1	10	0.12	0.943	0.955	0.863	0.643	0.851
D2	5	0.14	0.932	0.930	0.837	0.596	0.824
D3	5	0.16	0.922	0.933	0.828	0.588	0.818
D4	5	0.18	0.933	0.943	0.829	0.582	0.822
D5	5	0.20	0.925	0.927	0.825	0.580	0.814

4.4 Per-Point Quality Map

The per-point quality map reveals the spatial distribution of distortion in a way that aggregate scores cannot. Coloring each scatter plot point by its individual Trustworthiness score on the viridis color scale, with point size also redundantly encoding quality, produces a clear visual separation between reliable and unreliable regions.

Points belonging to digit classes 0, 1, and 6 consistently show high per-point T and C scores, appearing as large, warm-colored points in both the baseline and optimized embeddings. These digits have distinctive visual forms, such as the closed oval of 0, the vertical stroke of 1, and the curled shape of 6, that do not overlap substantially with other classes in pixel space, and UMAP faithfully represents their proximity structure.

Points belonging to digit classes 3, 4, 8, and 9 show consistently lower per-point scores, appearing as smaller, cooler-colored points. The minimum per-point scores observed, $T =$

Table 4.3: Baseline vs. Best Configuration Found

Metric	**Baseline**	**Best (n=25, md=0.14)**
Trustworthiness (T)	0.941	0.938
Continuity (C)	0.955	0.950
Steadiness (S)	0.868	0.844
Cohesiveness (Co)	0.620	0.681
Composite	0.846	0.853

0.505 for a digit-7 point and $C = 0.440$ for another point in the overlap region, indicate that for some individual points, roughly half of their apparent proximity relationships in the scatter plot are false neighbors or half of their genuine neighbors are missing.

This per-point decomposition directly addresses a limitation of aggregate statistics. The global T of 0.9281 obscures the fact that, for some points, the local neighborhood structure is highly misleading. An analyst examining the scatter plot and trusting the aggregate score might assume that about 93% of all proximity relationships are reliable. The per-point quality map reveals that this reliability is non-uniformly distributed. Most of the scatter plot is highly reliable, but specific regions corresponding to visually similar digit pairs are substantially less so.

4.5 Performance Benchmarks

Table 4.4 reports computation times for each metric on the MNIST-200 dataset, measured using `performance.now()` across three repeated runs.

Table 4.4: Metric Computation Times on MNIST-200 ($n = 200$, $d = 784$)

Metric	**Min**	**Mean**	**Max**
Trustworthiness	17.6 ms	24.2 ms	36.8 ms
Continuity	17.1 ms	18.1 ms	19.4 ms
Steadiness + Cohesiveness	221.2 ms	232.8 ms	247.6 ms

Trustworthiness and Continuity take approximately 18 to 24 ms each. Both require computing two full $n \times n$ pairwise distance matrices, one for the original high-dimensional space and one for the 2D embedding. This is an $O(n^2 \cdot d)$ operation that represents the

computational bottleneck. For $n = 200$ and $d = 784$, this involves approximately 20,000 distance computations per matrix, each over a 784-dimensional vector.

Steadiness and Cohesiveness are substantially more expensive at approximately 233 ms total. These metrics are computed jointly in a single call, and their higher cost comes from the randomized cluster extraction procedure. HDBSCAN is run on the embedding, and for each identified cluster, a random walk process samples member points over 150 iterations to measure cluster cohesion. This procedure is inherently more computation-intensive than the rank-based neighborhood calculations used by T and C.

All four metrics complete within approximately 280 ms total on MNIST-200, which is well within the interactive threshold for a browser-based tool.

4.6 Interactive Parameter Tuning Interface

To make the MNIST benchmark results tangible and to demonstrate the full ZADU.js pipeline in an explorable form, we developed a browser-based interactive parameter tuning interface (`visualize-tuning-digits.html`). The application integrates `druid.js` UMAP, ZADU.js metrics, and D3.js rendering into a single self-contained HTML file, requiring no server, build step, or installation beyond opening the file in a browser.

Layout and interaction model. The interface is organized as a vertical stack of panels. At the top, a Quality Overview panel displays the current per-metric scores and a color-coded severity badge, excellent, good, fair, or poor, for each metric. Below that, a UMAP Parameters panel exposes two sliders one for **n_neighbors** (range 5 to 50) and one for **min_dist** (range 0.01 to 0.50). Adjusting the sliders and clicking "Re-run UMAP" triggers a full UMAP re-projection in the browser using the new parameter values, followed by recomputation of Trustworthiness and Continuity. The result appears in the right panel of a side-by-side scatter plot comparison, positioned next to the fixed baseline embedding at **n_neighbors** $= 15$, **min_dist** $= 0.10$.

Digit thumbnail rendering. Each scatter plot point is rendered as a small image of the handwritten digit rather than a simple marker. The image is generated in-browser from the raw 784-dimensional pixel data using an HTML canvas element, producing a 28×28

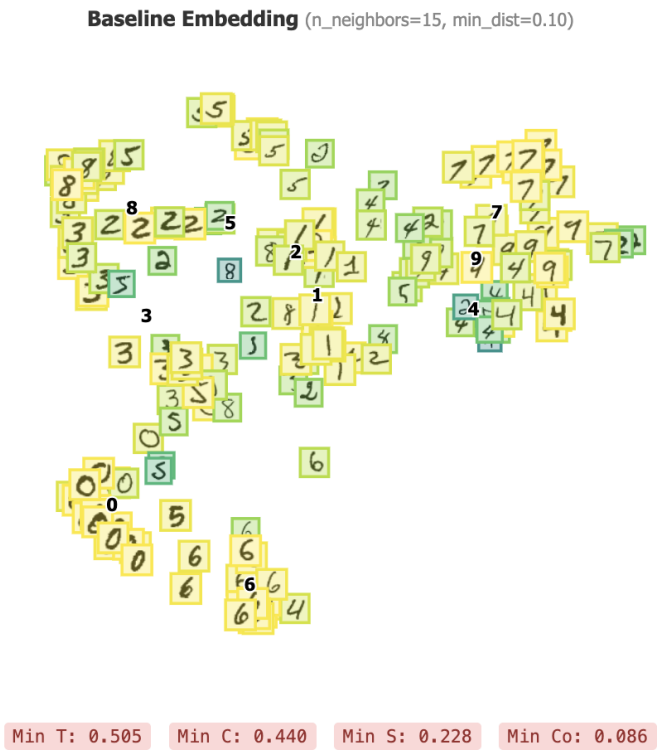

Figure 4.1: UMAP baseline embedding on 200 MNIST samples with $\texttt{n_neighbors} = 15$ and $\texttt{min_dist} = 0.1$.

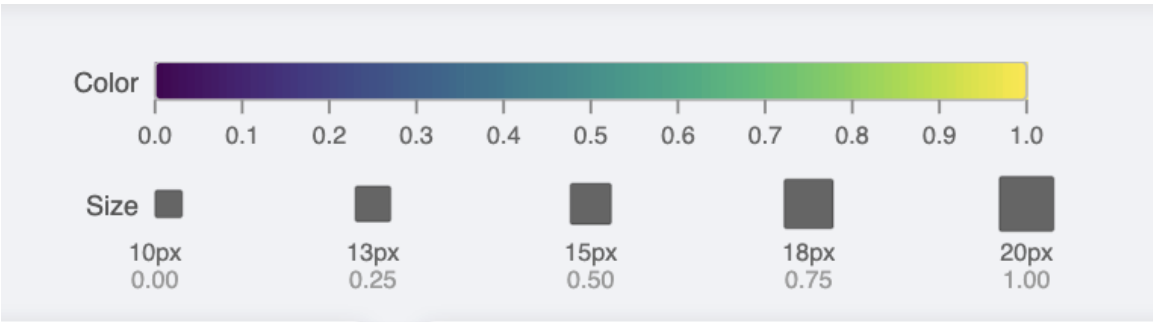

Figure 4.2: Color and size legend.

grayscale thumbnail for each of the 200 sample points. The border color encodes the digit class, and the fill overlay encodes the per-point quality score for the selected metric. Both color and size vary with quality, so high-quality points appear larger and more visually prominent.

Per-point quality coloring. A control panel allows the user to switch between coloring by cluster, using digit class, and coloring by metric score. When coloring by metric score, the user can select one of the four metrics, T, C, S, or Co, and choose a color scheme such as grayscale, viridis, spectral, turbo, or blue-red. A color and size legend is rendered dynamically, showing a gradient bar and sample markers at representative score values. This enables interpretation of the quality map without requiring interaction with individual points.

Hover and neighbor highlighting. Hovering over a point reveals a tooltip containing the point index, digit class, and all available per-point ZADU scores. To address visual overlap, a D3 force simulation temporarily separates nearby points on hover and restores their positions on mouse exit. Clicking a point highlights it and its seven nearest neighbors in the original high-dimensional space across both panels, allowing direct comparison between the baseline and updated embeddings.

Metrics comparison table. Below the scatter plots - as seen in Figure 4.3 a comparison table displays the global T and C scores for both embeddings, along with the change in each score. Improvements are shown in green, regressions in red, and negligible changes in gray. The table also reports minimum per-point scores, which serve as a sensitive indicator of worst-case distortion. Quality badges annotate each value for quick interpretation.

Implementation notes. Trustworthiness and Continuity are computed in-browser on each UMAP run using the same implementation as in the Node.js benchmark. Steadiness and Cohesiveness rely on HDBSCAN and are therefore precomputed offline and loaded from a dataset file. The metric implementations themselves run in-browser, as established in Section 4.5; offline precomputation is a current limitation of the HDBSCAN dependency, not of ZADU.js's architecture. The high-dimensional pairwise distance matrix is computed once on page load and cached, avoiding repeated $O(n^2 \cdot d)$ computation. UMAP re-runs take approximately 3 to 5 seconds for 350 epochs, and metric recomputation adds approximately 20 to 50 ms.

Figure 4.4 shows the interface with an optimized embedding (**n_neighbors** $= 12$, **min_dist** $= 0.08$) displayed in the right panel, colored by per-point Trustworthiness using the viridis scale. Digit classes 0, 1, and 6 appear large and bright, consistent with their high per-point scores. Classes 4 and 9 appear smaller and darker, with overlapping spatial regions, confirming the Cohesiveness limitations identified earlier.

Metrics Comparison

Metric	Baseline	New	Change	Min (Baseline)	Min (New)	Min Change
Trustworthiness	0.9305 excellent	0.9383 excellent	+0.0078	0.6652 poor	0.3107 poor	-0.3545
Continuity	0.9505 excellent	0.9523 excellent	+0.0018	0.4308 poor	0.5616 poor	+0.1308
Steadiness	0.7913 poor	--	--	0.2279 poor	--	--
Cohesiveness	0.6432 poor	--	--	0.0860 poor	--	--

Figure 4.3: Metrics comparison table showing baseline and optimized scores with per-metric changes.

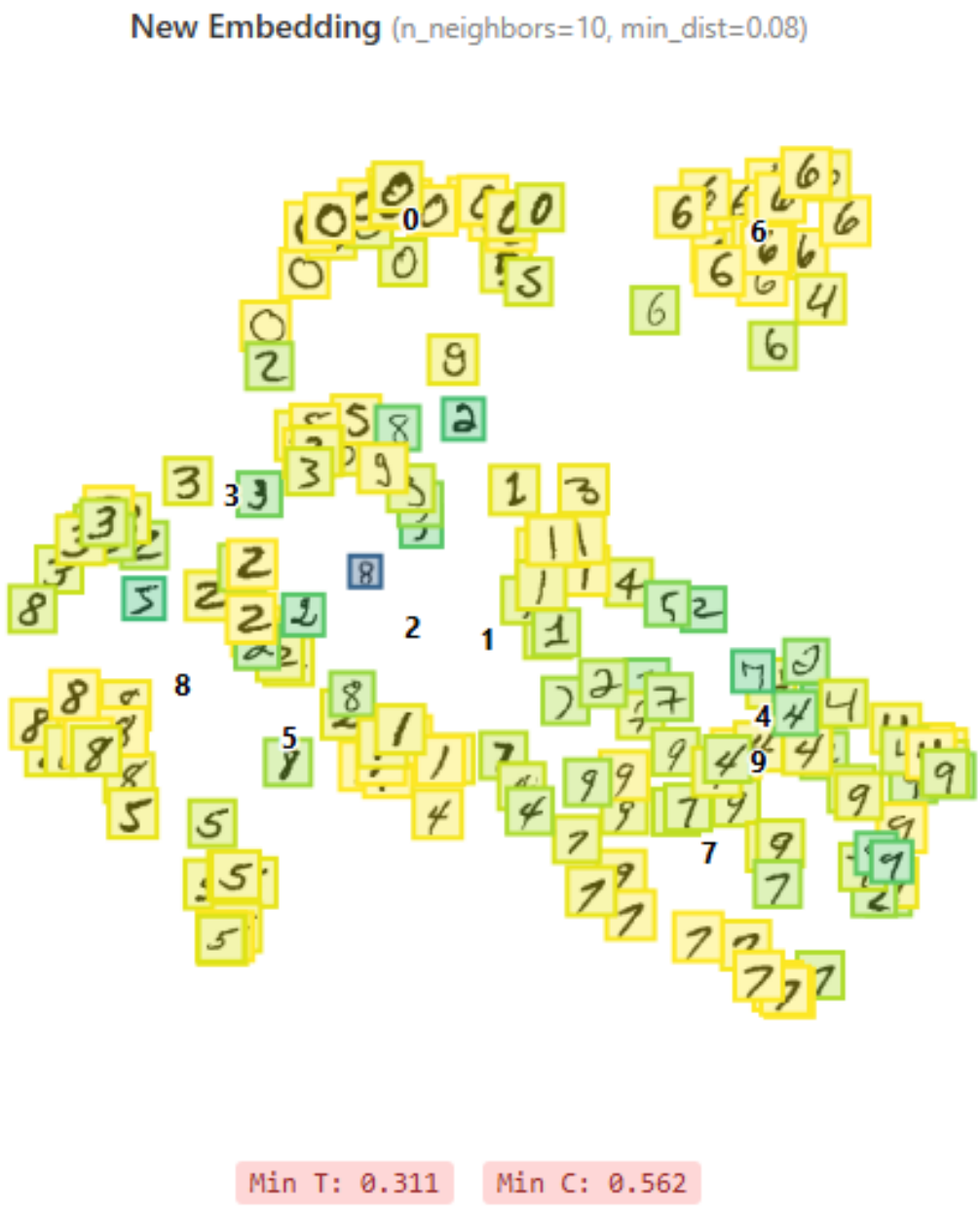

Figure 4.4: UMAP embedding recomputed with **min_dist** $= 0.08$ and **n_neighbors** $= 10$, colored using the viridis scale.

Chapter 5

Conclusion

A persistent obstacle in dimensionality reduction is that analysts have no reliable way to determine how much to trust what they see. A scatter plot of high-dimensional data appears identical whether the 2D projection faithfully preserves neighborhood structure or silently introduces false neighbors and omits genuine ones. Aggregate quality scores exist in the research literature, but they reduce the entire embedding to a single number, concealing the spatial distribution of distortion. This thesis addresses that gap with localized quality assessment that registers directly onto the scatter plot, paired with an interactive parameter tuning interface that makes the effect of each configuration measurable rather than a matter of visual judgment.

This work addresses that gap through the combination of four complementary quality metrics with an interactive browser-native interface that enables analysts to see distortion and reason about it within a single session. ZADU.js computes Trustworthiness and Continuity, point-level metrics measuring false and missing neighbors, respectively, as well as Steadiness and Cohesiveness, cluster-level metrics measuring how reliably cluster boundaries are preserved. The per-point quality map encodes these scores as a color overlay on the scatter plot, making the spatial structure of distortion immediately visible. The interactive parameter tuning application extends this further: analysts adjust n neighbors and min dist with sliders and observe quality scores update in real time alongside two synchronized scatter plots, one for the previous configuration and one for the current configuration.

The MNIST benchmark demonstrates what this combination reveals in practice. Indi-

vidual digit classes differ substantially in projection fidelity. Digits 0, 1, and 6 are reliably projected, while digits 3, 4, 8, and 9, whose pixel geometries overlap in the original 784-dimensional space, show consistently lower per-point quality. A global Trustworthiness score of 0.928 obscures this variation entirely. The per-point quality map makes it visually explicit, enabling the analyst to identify which regions of the scatter plot to trust and which to question.

The directional parameter sweep reveals findings that purely visual judgment would not surface. Across 20 UMAP configurations spanning four search directions, increasing both `n_neighbors` and `min_dist` together achieved the best composite score of 0.853 and the highest Cohesiveness of 0.681. The conventional recommendation for low Cohesiveness is to increase `n_neighbors` while decreasing `min_dist`. Direction C, which follows this rule, achieved a best composite score of 0.851 and Cohesiveness of 0.670. The gap is small but consistent.

More importantly, the sweep reveals a deeper result: low Cohesiveness on MNIST is a property of the data itself rather than a correctable parameter deficiency. Digit classes that overlap in 784-dimensional pixel space will overlap in any 2D projection. An analyst attempting to improve Cohesiveness through parameter tuning would be working against the structure of the data, and the per-metric quality scores make this limitation explicit in a way that visual inspection alone cannot.

Quality assessment should be a first-class component of any dimensionality reduction visualization pipeline rather than an optional post hoc step. When quality information is spatially aligned with the embedding, computed automatically, and available within the same interactive session as parameter adjustment, it changes how analysts interpret and act on projections. ZADU.js makes this mode of analysis practical in the browser.

Chapter 6

Future Work

The current implementation of ZADU.js establishes browser-native dimensionality reduction quality assessment as both feasible and useful, but several directions for extension remain.

Scalability. The pairwise distance computation underlying Trustworthiness and Continuity scales as $O(n^2 \cdot d)$, where n is the number of points and d is the dimensionality. At $n = 200$ and $d = 784$, this completes in approximately 24 ms. At $n = 2{,}000$, the computation would take approximately 100 times longer, or about 2.4 seconds, which remains borderline interactive. At $n = 10{,}000$, it becomes impractical for a browser environment. Approximate nearest-neighbor methods, such as Hierarchical Navigable Small World graphs (HNSW), could reduce this to $O(n \log n)$ with bounded approximation error, enabling ZADU.js to scale to datasets of tens of thousands of points while maintaining interactive performance.

WebWorker parallelization. Currently, all metric computations run on the browser's main thread, which blocks the user interface during execution. For larger datasets, this would make the quality map unresponsive. Moving metric computations into a WebWorker thread would allow the interface to remain interactive during computation, with progress updates streamed back to the main thread.

Per-class quality breakdowns. The current per-point visualization encodes quality as a continuous variable. For labeled datasets, such as MNIST or the VIS paper corpus with topic annotations, it would be more informative to aggregate per-point scores by class

or cluster and report class-level statistics. This would allow analysts to identify which semantic groups are systematically underrepresented without inspecting individual points.

Formal usability study. The usability study protocol part of this project could be a 60-minute moderated think-aloud evaluation of the ZADU.js parameter tuning interface with 10 to 14 participants across four user profiles: dimensionality reduction practitioners, visualization researchers, domain scientists, and machine learning engineers. Conducting this study would provide empirical evidence for the claim that per-point quality visualization changes how analysts interpret and interact with dimensionality reduction projections, moving the current observation from an informal finding to a validated result.

Real-time quality feedback during UMAP optimization. In the current implementation, quality metrics are computed after UMAP has completed optimization. In principle, UMAP's optimization process could be instrumented to expose intermediate embeddings at each epoch. ZADU.js could compute quality scores on these intermediate states and feed them back into the optimization process. This would enable joint optimization of embedding quality and UMAP's reconstruction objective, potentially identifying parameter configurations that the current sequential approach does not discover.

array url

Chapter 7

Appendix, Links, Code and Notebooks

Table 7.1: ZADU.js Resources and Links

Contribution	Link
ZADU.js npm package (metrics)	https://www.npmjs.com/package/zadu-js
ZADU.js GitHub repository	https://github.com/jonathantarun/zadu-js
MNIST demo (parameter tuning interface)	https://jonathantarun.github.io/zadu-js/examples/visualize-tuning-digits.html
VIS 2025 paper explorer	https://johnguerra.co/viz/ieeevis2025Papers/
Observable Demo for importing metrics	https://observablehq.com/@jonathan-tarun-rajasekaran/zadu-js-npm-package-demo

Bibliography

[1] Ehsan Amid and Manfred K Warmuth. Trimap: Large-scale dimensionality reduction using triplets. *arXiv preprint arXiv:1910.00204*, 2019.

[2] Gabriel Appleby, Mateus Espadoto, Rui Chen, Samuel Goree, Alexandru C Telea, Erik W Anderson, and Remco Chang. Hypernp: Interactive visual exploration of multidimensional projection hyperparameters. In *Computer Graphics Forum*, volume 41, pages 169–181. Wiley Online Library, 2022.

[3] Michaël Aupetit. Visualizing distortions and recovering topology in continuous projection techniques. *Neurocomputing*, 70(7-9):1304–1330, 2007.

[4] Michael Bostock, Vadim Ogievetsky, and Jeffrey Heer. D^3 data-driven documents. *IEEE transactions on visualization and computer graphics*, 17(12):2301–2309, 2011.

[5] Lisha Chen and Andreas Buja. Local multidimensional scaling for nonlinear dimension reduction, graph drawing, and proximity analysis. *Journal of the American Statistical Association*, 104(485):209–219, 2009.

[6] Benoît Colange, Laurent Vuillon, Sylvain Lespinats, and Denys Dutykh. Interpreting distortions in dimensionality reduction by superimposing neighbourhood graphs. In *2019 IEEE Visualization Conference (VIS)*, pages 211–215. IEEE, 2019.

[7] Rene Cutura, Christoph Kralj, and Michael Sedlmair. Druid js—a javascript library for dimensionality reduction. In *2020 IEEE Visualization Conference (VIS)*, pages 111–115. IEEE, 2020.

[8] Mateus Espadoto, Rafael M Martins, Andreas Kerren, Nina ST Hirata, and Alexandru C Telea. Toward a quantitative survey of dimension reduction techniques. *IEEE transactions on visualization and computer graphics*, 27(3):2153–2173, 2019.

[9] Rebecca Faust, David Glickenstein, and Carlos Scheidegger. Dimreader: Axis lines that explain non-linear projections. *IEEE transactions on visualization and computer graphics*, 25(1):481–490, 2018.

[10] Hyeon Jeon, Michaël Aupetit, Soohyun Lee, Kwon Ko, Youngtaek Kim, Ghulam Jilani Quadri, and Jinwook Seo. Distortion-aware brushing for reliable cluster analysis in multidimensional projections. *IEEE Transactions on Visualization and Computer Graphics*, 2025.

[11] Hyeon Jeon, Aeri Cho, Jinhwa Jang, Soohyun Lee, Jake Hyun, Hyung-Kwon Ko, Jaemin Jo, and Jinwook Seo. Zadu: A python library for evaluating the reliability of dimensionality reduction embeddings. In *2023 IEEE Visualization and Visual Analytics (VIS)*, pages 196–200. IEEE, 2023.

[12] Hyeon Jeon, Hyung-Kwon Ko, Jaemin Jo, Youngtaek Kim, and Jinwook Seo. Measuring and explaining the inter-cluster reliability of multidimensional projections. *IEEE Transactions on Visualization and Computer Graphics*, 28(1):551–561, 2021.

[13] Hyeon Jeon, Kwon Ko, Soohyun Lee, Jake Hyun, Taehyun Yang, Gyehun Go, Jaemin Jo, and Jinwook Seo. Umato: Bridging local and global structures for reliable visual analytics with dimensionality reduction. *IEEE Transactions on Visualization and Computer Graphics*, 2025.

[14] Hyeon Jeon, Hyunwook Lee, Yun-Hsin Kuo, Taehyun Yang, Daniel Archambault, Sungahn Ko, Takanori Fujiwara, Kwan-Liu Ma, and Jinwook Seo. Unveiling high-dimensional backstage: A survey for reliable visual analytics with dimensionality reduction. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, pages 1–24, 2025.

[15] Samuel Kaski, Janne Nikkilä, Merja Oja, Jarkko Venna, Petri Törönen, and Eero Castrén. Trustworthiness and metrics in visualizing similarity of gene expression. *BMC bioinformatics*, 4(1):48, 2003.

[16] Joseph B Kruskal. Multidimensional scaling by optimizing goodness of fit to a non-metric hypothesis. *Psychometrika*, 29(1):1–27, 1964.

[17] John A Lee and Michel Verleysen. Quality assessment of dimensionality reduction: Rank-based criteria. *Neurocomputing*, 72(7-9):1431–1443, 2009.

[18] John A Lee and Michel Verleysen. Scale-independent quality criteria for dimensionality reduction. *Pattern Recognition Letters*, 31(14):2248–2257, 2010.

[19] Sylvain Lespinats and Michaël Aupetit. Checkviz: Sanity check and topological clues for linear and non-linear mappings. In *Computer Graphics Forum*, volume 30, pages 113–125. Wiley Online Library, 2011.

[20] Joshua Lochner. Transformers. js: State-of-the-art machine learning for the web, 2023.

[21] Wouter Lueks, Bassam Mokbel, Michael Biehl, and Barbara Hammer. How to evaluate dimensionality reduction?-improving the co-ranking matrix. *arXiv preprint arXiv:1110.3917*, 2011.

[22] Rafael Messias Martins, Danilo Barbosa Coimbra, Rosane Minghim, and Alexandru C Telea. Visual analysis of dimensionality reduction quality for parameterized projections. *Computers & Graphics*, 41:26–42, 2014.

[23] Leland McInnes, John Healy, and James Melville. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018.

[24] Luis Gustavo Nonato and Michael Aupetit. Multidimensional projection for visual analytics: Linking techniques with distortions, tasks, and layout enrichment. *IEEE Transactions on Visualization and Computer Graphics*, 25(8):2650–2673, 2018.

[25] Shivam Raval, Fernanda Viégas, and Martin Wattenberg. Hypertrix: An indicatrix for high-dimensional visualizations. In *2024 IEEE Visualization and Visual Analytics (VIS)*, pages 1–5, 2024.

[26] Jaume Ros, Alessio Arleo, and Fernando Paulovich. Mind the gaps: Measuring visual artifacts in dimensionality reduction. *arXiv preprint arXiv:2511.14544*, 2025.

[27] Arvind Satyanarayan, Dominik Moritz, Kanit Wongsuphasawat, and Jeffrey Heer. Vega-lite: A grammar of interactive graphics. *IEEE transactions on visualization and computer graphics*, 23(1):341–350, 2016.

[28] Christin Seifert, Vedran Sabol, and Wolfgang Kienreich. Stress maps: Analysing local phenomena in dimensionality reduction based visualizations. In *International Symposium on Visual Analytics Science and Technology, EuroVAST 2010*. Eurographics Association, 2010.

[29] Warren S Torgerson. Multidimensional scaling: I. theory and method. *Psychometrika*, 17(4):401–419, 1952.

[30] Laurens Van Der Maaten, Eric O Postma, H Jaap Van Den Herik, et al. Dimensionality reduction: A comparative review. *Journal of Machine Learning Research*, 10(1):1–41, 2009.

[31] Jarkko Venna and Samuel Kaski. Neighborhood preservation in nonlinear projection methods: An experimental study. In *International conference on artificial neural networks*, pages 485–491. Springer, 2001.

[32] Jarkko Venna and Samuel Kaski. Local multidimensional scaling. *Neural Networks*, 19(6-7):889–899, 2006.

[33] Jarkko Venna, Jaakko Peltonen, Kristian Nybo, Helena Aidos, and Samuel Kaski. Information retrieval perspective to nonlinear dimensionality reduction for data visualization. *Journal of Machine Learning Research*, 11(2), 2010.

[34] Yingfan Wang, Haiyang Huang, Cynthia Rudin, and Yaron Shaposhnik. Understanding how dimension reduction tools work: an empirical approach to deciphering t-sne,

umap, trimap, and pacmap for data visualization. *Journal of Machine Learning Research*, 22(201):1–73, 2021.